

Ans: The fibonacci sequence is defined by the following the first 2 values in the sequence are 1. Every subsequent value is the sum of the 2 values preceding. fibonacci sequence?

Algorithm:

Step 1: start

Step 2: first take the public class of the fibonacci

Step 3: Now add the if condition to return the n value

Step 4: Recursive fibonacci with $(n-2)$

Step 5: use iterative fibonacci to return the prev number value

Step 6: Return the fibonacci value

Step 7: print the function using iterative function

Step 8: stop

Source code:

Public class fibonacci {

public static int recursive fibonacci

if ($n \leq 1$) {

return n;

}

else {

return recursive fibonacci ($n-1$) +

recursive fibonacci ($n-2$);

}

}

Public static int iterative fibonacci

(int n) {

if ($n \leq 1$) {

return n;

}

int prev1 = 1; int

prev2 = 1; int fib = 0;

for (int i = 2; i $\leq$ n; i++) {

fib = prev1 + prev2;

prev2 = prev1;

prev1 = fib;

return fib;

}

Public static void main(String[] args) {

int n=5;

System.out.println("Using Recursive
function");

System.out.println("Fibonacci (" + n + ")
= " + recursiveFibonacci(n));

System.out.println("Using Iterative
function");

System.out.println("Fibonacci (" + n + ") =

using iterative fibonacci);

}

}

Output:

using Recursive function:

$$\text{fibonacci}(5) = 5$$

using Iterative function:

$$\text{fibonacci}(5) = 5$$

Result: The Java program that uses both recursive and fibonacci sequence is successfully executed

Aim: write a Java program that prompts the users for an Integer and then prints out all prime numbers upto Integer

Algorithm:

Step 1: Start

Step 2: Import the Java util Scanner

Step 3: use boolean isPrime with the Integer number

Step 4: use if condition ($n \leq 1$) and get the return value

Step 5: And if condition ($n \leq 3$) and get the return value

Step 6: Now use for loop and Integer $i = 5$

Step 7: Now Scanner of the Java file

Step 8: print the prime numbers

Step 9: Stop

Source code

Empost Java • Util • SC

Public class prime Number Finder {

Public static boolean is prime

(int number) {

if (number <= 1) {

return false;

}

if (number <= 3) {

return true;

}

if (number % 2 == 0 || number % 3 == 0)

{

return false;

}

for (int i = 5; i * i <= number; i += 5) {

if (number % i == 0 || number % (i + 2) == 0)

return false;

}

}

return true;

}

```
public static void main (String[] args) {  
    Scanner scanner = new Scanner(System.in);  
    System.out.println(  
        "Enter a number to find prime numbers upto it:");  
    int n = scanner.nextInt();  
    System.out.println("Prime numbers upto  
        " + n + " are:");  
    for (int i = 2; i <= n; i++) {  
        if (isPrime(i)) {  
            System.out.print(i + " ");  
        }  
    }  
    scanner.close();  
}
```

Output:

Enter a positive integer: 20
prime numbers up to 20 are:

2 3 5 7 11 13 17 19

Result: The Java program that uses
primality test point out all the prime
numbers is successfully executed.

Aim: Write a Java program that prints all real solutions to the quadratic equation $ax^2 + bx + c = 0$. Read in a, b, c and use the quadratic formula. If the $b^2 - 4ac$ is negative, all no real solutions!

Algorithm:

- Step 1: Start
- Step 2: First import Java util Scanner
- Step 3: Now use public class quadratic equation
- Step 4: Enter the co-efficient values of a
- Step 5: Double discriminate the $(b*b) - (4*a*c)$
- Step 6: Now double root and Math.sqrt
- Step 7: print the Root value and solution
- Step 8: STOP

Source code:

```
import java.util.Scanner;

public class QuadraticEquation {

    public static void main(String[] args) {

        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the co-efficients:");

        double discriminant = (b*b) - (4*a*c);

        if (discriminant > 0) {

            double root1 = (-b + Math.sqrt(discriminant)
                / (2*a));

            double root2 = (-b - Math.sqrt(discriminant)
                / (2*a));

            System.out.println("Real Solutions:");
            System.out.println("Root 1: " + root1);
            System.out.println("Root 2: " + root2);
        }

        else if (discriminant == 0)
```

```
double root = -b / (2 * a);  
system.out.println("One real solution");  
system.out.println("Root : " + root);  
}  
else {  
    system.out.println("No real solutions  
Discriminate is negative");  
}  
scanner.close();  
}
```

output:

Enter the Coefficient $a = 1$

Enter the Co-efficient $b = 6$

Enter the Co-efficient $c = 1$

Real solutions

Root 1: -3.1715728752538097

Root 2: -8.82842712474619

Result: Above program is successfully executed

Aim: write a Java program to create an abstract class named shape that contains two attributes and an empty method pointArea(). Each of the classes contains only the method pointArea() that points that area over shape.

Algorithm:

Step 1: Start

Step 2: First import the Java util package

Step 3: Now abstract the class Shape

Step 4: get the class Rectangle extends shape

Step 5: point the area of the double x, double y.

Step 6: Now point the area of the triangle

Step 7: class Abstract Rectangle new

Step 8: As same for circle class abstract

Step 9: STOP

Source code:

```
import java.util.*;  
abstract class Shape  
{  
    int x, y;  
    abstract void pointArea(double x, double y);  
}  
class Rectangle extends Shape  
{  
    void pointArea(double x, double y)  
{  
        System.out.println("Area of triangle  
is  $x^2 + (0.5 \times x \times y)$ ");  
}  
}  
class abstract Demo  
{  
    public static void main (String[] args)  
    {  
        Rectangle r = new  
        Rectangle (); r.pointArea (2, 5);  
    }  
}
```

Expt.No. 04

Page No. 12

Date

Circle c = new Circle();

c.print Area (5, 5);

Triangle t = new Triangle();

t.print Area (2, 5);

3

3



Output:

Area of rectangle is : 10.0

Area of circle is : 78.53981633974483

Area of triangle is : 5.0

Result: Above program is successfully executed.

Aim: write a Java program that implements a multithread application that has three threads generate random Integer every 1 value is even, second thread computes the square of the value is odd the third thread cube of the number.

Algorithms:

Step 1: Start

Step 2: First import the Java Util Random

Step 3: square the extends the thread of the program

Step 4: Now print the cube of the value

Step 5: class the extends thread of the public void run of the value

Step 6: Random the Integer generated of the square

Step 7: Add the class multithread Demo public static void

Step 8: STOP

Source code:

```
import java.util. Random; class  
square extends Thread
```

```
{  
    int x;  
    square (int n)
```

```
{  
    x=n;
```

```
}  
    public void run()
```

```
{  
    int sqx = x * x;
```

```
    System.out.println ("Cube of " + x + " is " + sqx);  
}
```

```
}  
class Number extends Thread
```

```
{  
    public void run()
```

```
{  
        Random random = new Random();
```

```
        for (int i=0; i<10; i++)
```

```
{  
            int randomInteger = new Random()
```

```
                .setSeed (1) .nextInt();
```

```
            generator = randomInteger;
```

```
            square s = new square (randomInteger);
```



```

Cube c=new Cube(Random Integer);
c.start();

```

```

try {
    Thread.sleep(1000);
}

```

Generated random number is then passed as argument to square and cube threads output varies each time a program is executed. */

```

}
catch (InterruptedException ex) {
    System.out.println(ex);
}

```

```

}
}
}
}

```

```

}
public class multithread Demo {
    public static void main (String args[])
    {

```

```

        number n=new number();

```

```

        n.start();

```

```

    }

```

```

}

```

Output:

5.

Random Integer generator : 93

Square of 93 = 8649

Random Integer generated : 40

Square of 40 = 1600

Cube of 40 = 64000

Random Integer generated : 57

Square of 57 = 3249

Cube of 57 = 185193

Result: Above program is successfully executed.

Aim: Write a program to create Interface A in this interface we have two methods meth1 and meth2. Implements this interface in another class named myclass.

Algorithm:

Step 1: start

Step 2: Take Interface A to get method 1 & 2

Step 3: Now extend the A with Interface B

Step 4: Take class as implement B

Step 5: print the implement method 1

Step 6: Now take class IF extend

Step 7: take object as method 1 & 2 & 3

Step 8: STOP

Source Code:

Interface A

```
{  
    void meth1();  
    void meth2();  
}
```

Interface B extends A

```
{  
    void meth3();  
}
```

Class myday implements B

```
{  
    public void meth1()  
    {  
        System.out.println("Implement meth1");  
    }  
    public void meth2()  
    {  
        System.out.println("Implement meth2");  
    }  
    public void meth3()  
    {  
        System.out.println("Implement meth3");  
    }  
}
```

class If extend

§ public static void main (String args)

§

try class ob=new myclass ();

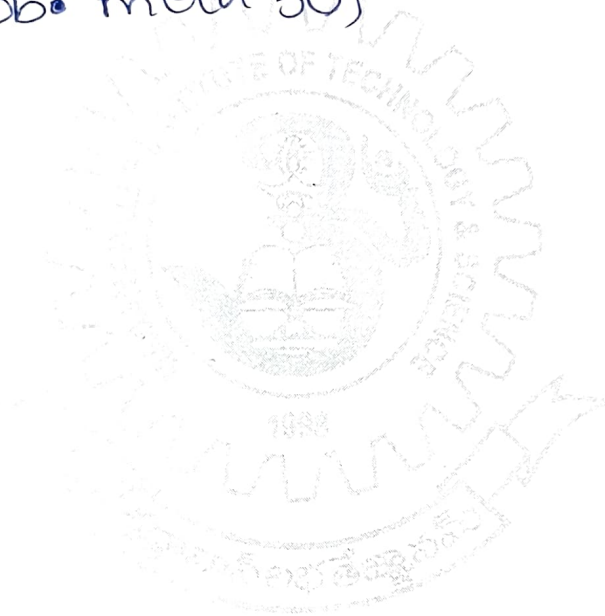
ob.meth1();

ob.meth2();

ob.meth3();

§

§



Output

Implement meth 1()

Implement meth 2()

Implement meth 3()

Results - Above the program is successfully
executed

Aim: write a program to create Interface named test. In this Interface the member function is Square. Implement this in to test In this class the object of arithmetic

Algorithm:

Step 1: Start

Step 2: Take public class Interface Test

Step 3: Add the class as arithmetic Test

Step 4: Take class implement method Square Value

Step 5: Now class to Test Interface of the program.

Step 6: call the Test Int from the main method

Step 7: create the object of the test interface

Step 8: print the Arithmetic method Square

Step 9: Stop

Source code:

```
public interface Test
```

```
{
```

```
    public int square(int a);
```

```
}
```

```
class Arithmetic implements Test
```

```
{
```

```
    int s=0;
```

```
    public int square(int b)
```

```
{
    System.out.println("Inside arithmetic  
class - implemented method square");
```

```
    System.out.println("square of "+a+" is");
```

```
}
```

```
void arithmetic)
```

```
{
```

```
    System.out.println("Inside a method  
class Arithmetic");
```

```
}
```

```
}
```

```
class TestInt
```

```
{
```

```
public static void main (String a[])
```

```
{
```

```
System.out.println("calling from to  
Test int class main method");
```

```
Test = new Arithmetic();
```

```
System.out.println("=====");
```

```
System.out.println("created object of test  
Interface reference Arithmetic class");
```

```
System.out.println("this object can't  
call as meth method of Arithmetic class")
```

```
System.out.println("=====")
```

```
test.sqaure();
```

```
System.out.println("=====");
```

```
}
```

```
}
```

Output:

calling from Test Int class main method

=====

Created object of test interface reference
arithmetic class

Hence arithmetic class method square called
This object can't call armeth method of

arithmetic class

=====

square of 4 is 16

=====

Result: The Above program is
Successfully executed

Aim: Write a Java program using package

i) Write a program to create a package named my pack and import it in code class

ii) Write a program to create a package name p1, and implement this package.

Algorithm:

Step 1: Start

Step 2: Take the package mypack

Step 3: Now take class circle with double

Step 4: Import the mypack from circle

Step 5: Now add the area of the circle

Step 6: As same for package P1

Step 7: Take public class ex and print the created and Hello

Step 8: Import package P1 and display the Value

Step 9: Stop

Source codes

Q) 8.1

package my pack;

class circle

{

double r;

void area();

{

System.out.println("Area of the Circle.

 $= 1 + (3.14 * r * r)$);

}

}

import my pack . circle;

class Eq

{
public static void main(String a[])

{

circle c = new circle();

c.area();

}

}

Q) 8.2

package P1

public class ex

```
    public exc()
    {
        System.out.println("created");
    }

    public void display()
    {
        System.out.println("Hello");
    }
}

import P1.*;
class ex1
{
    public static void main (String []a)
    {
        ex e = new ex();
        e.display();
    }
}
```

Output:

Area of the circle = 19.625

Area of the square = 6.25

Area of the Rectangle = 17.25

Result:—The given above the program
is successfully executed.

Aim: Write a Java program that reads on file name from the user then displays information about whether the file exists, readable, whether file length of the file in bytes?

Algorithm:

- Step 1: Start
- Step 2: Import Java IO file
- Step 3: Now take the class file demo
- Step 4: Take File name as read and write
- Step 5: If file is directory then list of file
- Step 6: Buffer the reader by with new Input stream.
- Step 7: Enter the file name
- Step 8: Stop

Source code:

```
import java.io.*;
```

```
class fileDemo
```

```
{
    public static void p(String str){
```

```
{
    System.out.println(str);
```

```
{
    public static void main (String args[])
```

```
{
    File f = new File(" ");
```

```
{
    if (f.exists())
```

```
{
    P(f.getName() + " is a file");
```

```
P(f.canRead() ? "is readable" : "is not readable");
```

```
P(f.canWrite() ? "is writeable" : "is not writeable");
```

```
P(f.length() + " bytes");
```

```
P(f.lastModified() + " last modified");
```

```
}
```

```
if (toDirectory())
```

```
{  
    P("get Name of directory");  
    P("List of files");
```

```
    String dir[] = f.list();
```

```
    P(dir[i]);
```

```
}
```

```
}
```

```
}
```

```
public class fileDetails
```

```
{  
    public static void main(String[]
```

```
args) throws IOException
```

```
{  
    BufferedReader br = new BufferedReader
```

```
(new InputStreamReader(System.in));
```

```
    System.out.println("Enter the file  
name");
```

```
    f.doAnalyze();
```

```
}
```

```
}
```

Output:

Enter the file name;

eg. save

eg. save is a file

is readable

is writable

file size: 125 Bytes

file last modified: 17 30 80 32 64 600

Result: The given file program is
Successfully executed.